



**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Gerência de Memória

Sistemas Operacionais - Laboratório 7

Curso: Engenharia de Telecomunicações
Professor: Rogerio Pereira Junior

Aluno:
Victor E. de L. Guerra

02/06/2026

Sumário

1	Conhecendo a memória do sistema	2
1.1	Comando <code>free -h</code>	2
1.2	O que é memória SWAP?	2
2	Informações Detalhadas da Memória	2
2.1	Comando <code>cat /proc/meminfo</code>	2
2.2	O que representa a memória cache?	2
2.3	Por que o Linux utiliza parte da RAM como cache?	2
3	Monitorando Processos	3
3.1	Comando <code>top</code> e os 5 processos que mais consomem memória	3
3.2	Qual processo consome mais memória?	3
3.3	Esse processo pertence ao sistema operacional ou ao usuário?	3
4	Criando Consumo Artificial de Memória	3
4.1	Quanto a memória utilizada aumentou?	3
4.2	O processo Python apareceu entre os maiores consumidores de memória?	3
4.3	O que aconteceu com a quantidade de memória livre?	3
5	Mapeando a Hierarquia de Memória (Caches e RAM)	3
5.1	Qual é o tamanho dos caches L1, L2 e L3 da sua máquina? Explique, com base na teoria da pirâmide de memória, por que o cache L1 é drasticamente menor do que a memória RAM instalada.	4
6	Explorando a Memória Virtual	4
6.1	Houve movimentação entre RAM e swap?	4
6.2	O sistema apresentou sinais de falta de memória?	5
7	Explorando Endereços Lógicos	5
8	Explorando Espaços de Endereçamento	5

1 Conhecendo a memória do sistema

1.1 Comando `free -h`

Após rodar o comando `free -h`, encontrei os seguintes valores:

- Memória RAM total: **16 GB**
- Memória RAM utilizada: **4.6 GB**
- Memória RAM livre: **8.5 GB**
- Espaço total de Swap: **4.8 GB**

1.2 O que é memória SWAP?

A memória **SWAP** é uma área do armazenamento (HD ou SSD) que funciona como uma extensão da memória RAM, ou seja, quando a memória RAM começa a ficar cheia o sistema pode mover para a memória SWAP (HD ou SSD do sistema) dados que não estão sendo usados no momento, liberando espaço na RAM para programas que precisam de acesso rápido.

2 Informações Detalhadas da Memória

2.1 Comando `cat /proc/meminfo`

Após rodar o comando `cat /proc/meminfo`, pude verificar as seguintes informações:

- **MemTotal:** 15726908 Kb
- **MemFree:** 8270596 Kb
- **Buffers:** 169640 Kb
- **Cached** 3153900 Kb
- **SwapTotal:** 4729852 Kb
- **SwapFree:** 4729852 Kb

2.2 O que representa a memória cache?

A memória cache representa uma memória de alta velocidade utilizada para armazenar temporariamente dados e instruções acessados com frequência pelo processador, reduzindo o tempo médio de acesso à memória e aumentando o desempenho do sistema.

2.3 Por que o Linux utiliza parte da RAM como cache?

O Linux utiliza parte da RAM como cache para melhorar o desempenho do sistema. Ao armazenar em memória os arquivos e dados acessados recentemente, o sistema pode atendê-los mais rapidamente do que se precisasse buscá-los no disco rígido ou SSD, que é muito mais lento. Essa estratégia reduz o tempo médio de acesso à informação e aumenta a eficiência geral do sistema, mesmo que parte da RAM pareça "ocupada", pois o Linux libera essa memória para programas quando necessário.

3 Monitorando Processos

3.1 Comando `top` e os 5 processos que mais consomem memória

Utilizando o comando `top` pude analisar que os 5 processos que mais estavam consumindo memória estão representados na imagem abaixo:

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
4503	aluno	20	0	19,8g	834536	356524	S	8,0	5,3	10:13.47	firefox+
17877	aluno	20	0	3075604	523220	136824	S	0,3	3,3	0:48.77	Isolate+
16463	aluno	20	0	7181684	428080	150412	S	0,0	2,7	3:51.69	Isolate+
3174	aluno	20	0	1588400	362604	104788	S	0,0	2,3	0:06.16	gnome-s+
8749	aluno	20	0	2852260	352984	136344	S	0,0	2,2	0:17.16	Isolate+

Figura 1: 5 processos que mais estavam consumindo memória

3.2 Qual processo consome mais memória?

O processo com o PID **4053** é o que mais consumia memória, processo esse referente ao navegador Firefox.

3.3 Esse processo pertence ao sistema operacional ou ao usuário?

Pertence ao usuário (aluno).

4 Criando Consumo Artificial de Memória

Após realizar a execução dos scripts indicados no relatório para gerar um consumo artificial de memória e abrir outros dois terminais e rodar os comandos `free -h` e `top`, podemos fazer as análises.

4.1 Quanto a memória utilizada aumentou?

Após gerar o consumo artificial de memória, a memória utilizada subiu de **4.6 GB** para **12 GB** e a memória livre caiu para **501 MB**.

4.2 O processo Python apareceu entre os maiores consumidores de memória?

Sim, o processo python apareceu como o **MAIOR** consumidor de memória.

4.3 O que aconteceu com a quantidade de memória livre?

Ela caiu drasticamente, de **8.5 Gb** para apenas **501 Mb**

5 Mapeando a Hierarquia de Memória (Caches e RAM)

Após analisar as informações sobre os níveis de cache instalados no processador e entender melhor a geometria do cache L1, vendo seu tamanho e tipo, posso responder a questão:

5.1 Qual é o tamanho dos caches L1, L2 e L3 da sua máquina? Explique, com base na teoria da pirâmide de memória, por que o cache L1 é drasticamente menor do que a memória RAM instalada.

Os resultados obtidos indicam que a máquina possui:

- Cache L1 de dados (L1d): 192 KiB distribuídos em 6 instâncias, correspondendo a 32 KiB por núcleo.
- Cache L1 de instruções (L1i): 192 KiB distribuídos em 6 instâncias, correspondendo a 32 KiB por núcleo.
- Cache L2: 3 MiB distribuídos em 6 instâncias, correspondendo a 512 KiB por núcleo.
- Cache L3: 16 MiB compartilhados entre todos os núcleos.

A confirmação de que o cache L1 do núcleo analisado possui 32 KiB e é do tipo dados foi obtida pelos comandos:

```
cat /sys/devices/system/cpu/cpu0/cache/index0/size
32K
```

```
cat /sys/devices/system/cpu/cpu0/cache/index0/type
Data
```

Segundo a teoria da pirâmide de memória, quanto mais próxima a memória está do processador, menor é sua capacidade, porém maior é sua velocidade de acesso. O cache L1 é o nível mais próximo da CPU e precisa operar com latência extremamente baixa, geralmente em poucos ciclos de clock. Para atingir essa velocidade, ele é construído com tecnologias de hardware mais rápidas e mais caras por bit armazenado do que a memória RAM.

Por esse motivo, o cache L1 possui capacidade reduzida (32 KiB por núcleo nesta máquina), enquanto a memória RAM pode possuir vários gigabytes. A RAM é significativamente mais lenta, mas oferece grande capacidade a um custo menor. Assim, a hierarquia de memória busca equilibrar desempenho e custo: pequenas quantidades de memória muito rápida próximas ao processador (L1, L2 e L3) e grandes quantidades de memória mais lenta em níveis inferiores (RAM e armazenamento secundário).

6 Explorando a Memória Virtual

Após execução do comando `vmstat 2` e observar os campos `si` e `so`, consigo responder as seguintes questões:

6.1 Houve movimentação entre RAM e swap?

Não. As colunas `si` (swap-in) e `so` (swap-out) permanecem em 0 em todas as amostras coletadas, e a coluna `swpd` mantém-se igualmente em 0 durante todo o período. Isso indica que nenhuma página de memória foi transferida da RAM para a swap nem da swap de volta para a RAM, ou seja, não houve movimentação entre RAM e swap.

6.2 O sistema apresentou sinais de falta de memória?

Não. A coluna `free` permanece estável em torno de 15,2 GB de memória livre durante toda a amostragem, a swap não foi utilizada (`swpd`, `si` e `so` iguais a 0) e a fila de processos bloqueados (`b`) permanece em 0 em todas as linhas. A ausência de pressão sobre a swap e a grande quantidade de memória livre disponível mostram que o sistema não apresentou sinais de falta de memória.

7 Explorando Endereços Lógicos

O mapa de memória do processo (PID 3396) mostra com clareza a organização típica do espaço de endereçamento virtual de um processo Linux x86-64.

As cinco primeiras linhas correspondem ao próprio executável `endereco`, divididas em segmentos por permissão: o `.text` (código) aparece como `r-xp` (leitura/execução), enquanto os segmentos somente-leitura (`r-p`) contêm dados constantes e o segmento `rw-p` contém os dados graváveis — é nessa última região que reside a variável `global`, por ser uma variável global inicializada.

Logo abaixo está o `[heap]`, marcado como `rw-p`, que cresce sob demanda (via `brk/malloc`) — embora este programa não aloque memória dinâmica, o heap já é reservado.

No meio aparecem as bibliotecas compartilhadas: a `libc.so.6` e o carregador dinâmico `ld-linux-x86-64.so.2`, ambos seguindo o mesmo padrão de segmentação por permissões (código em `r-xp`, dados em `rw-p`). Note que estão em endereços altos (`0x7de9...`), separados do executável.

Por fim, no topo do espaço de endereços, está a `[stack]` (`rw-p`) — onde mora a variável `local`. Junto dela aparecem `[vvar]` e `[vdso]`, regiões especiais mapeadas pelo kernel para acelerar certas chamadas de sistema.

Um detalhe que confirma o ASLR (*Address Space Layout Randomization*): os endereços-base do executável, das bibliotecas e da `stack` são valores “aleatórios” e diferentes a cada execução. Daí também a enorme distância entre o endereço de `global` (na faixa baixa do executável, `0x6020...`) e o de `local` (na faixa altíssima da `stack`, `0x7fff...`), exatamente o que os dois `printf` do programa ilustram.

8 Explorando Espaços de Endereçamento

As duas instâncias do mesmo executável exibem endereços diferentes tanto para `global` quanto para `local`, embora rodem exatamente o mesmo código. Isso evidencia dois conceitos centrais de gerenciamento de memória.

Primeiro, cada processo possui seu próprio espaço de endereçamento virtual — os endereços impressos são lógicos (virtuais), não físicos. O mesmo valor virtual em processos distintos é traduzido pela MMU para *frames* físicos diferentes, de modo que os processos ficam isolados entre si.

Segundo, os valores diferentes entre as duas execuções são fruto do ASLR (*Address Space Layout Randomization*): o endereço-base do executável (`0x5a6f...` vs `0x618e...`) e o da `stack` (`0x7ffd...` vs `0x7ffe...`) são randomizados a cada carga do programa, dificultando ataques que dependem de endereços previsíveis. Note que os *offsets* baixos se mantêm (`...010` para `global` nas duas execuções), porque a posição relativa dentro do segmento não muda — apenas a base sorteada.